Simulador Steiner Testes.c474 linhas

C

claude estou com uma duvida tecnica extremamente dificil e nao sei dizer qual é a certa, estou usando AG para criar arquiteturas de redes FTTH, ai cada individuo é uma arvore contendo postes de passagem a OLT e os clientes né, ai eu to usando taxa de mutação de 5% 100 individuos e 2000 gerações. Meus professores disseram que uma taxa acima de 10% é muito alta, mas ai eu tenho uma duvida que eu n aprendi até agora, na mutação, eu muto os individuos ou os nós dos individuos? Ou seja, mutar 5% dos nós de cada individuo ou de 100 individuos 5% terao 1 nó mutado? Quando eu deixo com a primeira opção ele vai com melhores resultados, mas eu tenho medo de ficar muito aleatorio e menos genetico, e a segunda opção nao faz diferença, tipo assim a quantidade de metros da rede da primeira geração é a mesma da 2000 geração.

Explicando estagnação e propondo mutação por gene como padrão.

Boa pergunta — essa é uma confusão super comum, e o seu código já dá a resposta de qual das duas opções vocês estão implementando hoje.

O que o seu código faz agora

Olhando mutacao_arvore:

c

```
int chance = rand() % 100;

if (chance < taxa_mutacao_atual) {

  int no_mutado = ...

  ...

  ind->pai[no_mutado] = vizinhos_validos[sorteio]; // só 1 nó

}
```

Essa função é chamada **uma vez por indivíduo por geração**, e o if decide se **aquele indivíduo inteiro** sofre mutação. Quando sofre, só **um único nó** é alterado. Ou seja: hoje vocês estão na sua "segunda opção" — 5% dos 100 indivíduos (≈5 indivíduos) recebem 1 nó mutado, e os outros 95 não mudam nada.

Qual é o padrão "de livro" em AG

Na formulação clássica (Holland, Goldberg, etc.), a taxa de mutação é definida **por gene**, não por indivíduo. Cada gene (aqui, cada nó/aresta do indivíduo) tem uma probabilidade independente pm de mutar. É por isso que a

recomendação "acima de 10% é muito alta" faz sentido nessa definição: se cada um dos, digamos, 128 nós tem 10% de chance de mutar, isso já mexe em ~13 nós por indivíduo a cada geração — bastante disruptivo. Se a taxa fosse "por indivíduo" (sua opção 2), 10% seria irrisório, porque afeta pouquíssimos indivíduos e ainda por cima só 1 gene de cada.

Então **o seu professor provavelmente estava falando da convenção por gene**, e isso explica o motivo de:

1. A opção 1 (mutar 5% dos nós de cada indivíduo) dar resultados melhores — ela injeta bem mais diversidade genética real na população a cada geração.
2. A opção 2 (implementação atual) não mudar nada de uma geração pra outra — o efeito é estatisticamente desprezível: com 100 indivíduos e 5% de chance, você espera ~5 indivíduos afetados por geração, cada um com só 1 nó trocado, em uma árvore que pode ter dezenas ou centenas de nós. Combinado com elitismo (pop_nova[0] = pop[indice_rei]) e cruzamento que tende a reforçar arestas já presentes nos pais, a população converge rápido e fica presa (estagnação/convergência prematura), porque não há "combustível" de mutação suficiente pra escapar do ótimo local.

Recomendação

Implemente por gene (nó), mas com cuidado nesse encoding de árvore: cada mutação exige checar ciclo (forma_ciclo) e vizinhos válidos, então mutar muitos nós ao mesmo tempo no mesmo indivíduo pode ser mais caro computacionalmente e também mais "destrutivo" da estrutura da árvore (cada nó mutado pode invalidar/mudar a subárvore dependente dele). Por isso:

- Sim, use a probabilidade por nó (ex.: para cada nó i de 1 a nos_reais-1, sorteie rand()%100 < taxa_mutacao_atual e, se passar, tente mutar esse nó).
- Taxas típicas na literatura para mutação por gene ficam entre 0.1% e 5% quando o cromossomo é grande (dezenas/centenas de genes); acima de 10% por gene já é considerado agressivo na maioria dos problemas — o que bate com o que seus professores disseram.
- Se com 5% por nó os resultados ficarem bons e estáveis, não precisa necessariamente ir mais alto; o ideal é testar uma faixa (1%, 3%, 5%, 8%, 10%) e comparar convergência x custo final, exatamente como sua bateria de testes já faz.

Se quiser, posso reescrever mutacao_arvore para implementar a versão por nó (iterando sobre todos os nós do indivíduo em vez de decidir uma vez só por

indivíduo) — é uma mudança pequena e mantém toda a lógica de validação de ciclo que vocês já têm.

pode ser

Ajustando a função de mutação de árvore com probabilidade por nó.

Ajustando a função de mutação de árvore com probabilidade por nó.

Vou aplicar a mudança direto no seu arquivo, mantendo toda a lógica de validação de ciclo/vizinhos que já existe — só troca a decisão de "mutar o indivíduo inteiro (1 nó)" por "cada nó tem sua própria chance de mutar".

Executou 2 comandos, editou um arquivo, arquivos compartilhados

O que mudou:

- Separei a lógica de mutação de um nó específico em mutar_no(ind, no_mutado) — reaproveita exatamente a mesma checagem de vizinhos válidos e de ciclo que vocês já tinham.
- mutacao_arvore agora percorre **todos os nós** do indivíduo (de 1 até nos_reais-1, pulando a raiz que não tem pai) e sorteia, para cada um, se ele muta ou não, com a probabilidade taxa_mutacao_atual.

Com isso, uma taxa de 5% agora significa "cada nó tem 5% de chance de mutar", então em uma instância com 128 nós isso dá ~6 nós mutados por indivíduo por geração (em vez de 1 nó em ~5 indivíduos a cada geração). Isso deve resolver o problema de estagnação que você viu (custo igual da geração 1 até a 2000), porque agora toda geração injeta mutação de verdade em praticamente todos os indivíduos.

Sugestões para os próximos testes:

- Rode a mesma bateria de taxas (10/30/50/70/90%) com essa nova definição — provavelmente vocês vão ver que taxas acima de 10% ficam bem mais agressivas/instáveis agora, e que algo entre 1% e 5% já deve ser um ponto de equilíbrio melhor.
- Vale considerar taxas menores que 10 no vetor taxas_teste (tipo 1, 3, 5) já que agora o efeito por geração é bem mais forte.

Simulador steiner testes

C

Refinando crossover, mutação e critérios de validação do algoritmo.

Beleza, com população=100 e gerações=1000 confirmados, aqui vai a reescrita da Metodologia, refletindo o algoritmo real que você está usando hoje (o híbrido, com seleção ponderada por distância no cruzamento e na mutação — não mais o sorteio cego da versão anterior):

---

3. Metodologia

A execução inicia com a leitura dos dados da instância, registrando as coordenadas (x,y)(x,y) e a classificação dos vértices (OLT, postes e clientes). As vias viáveis são mapeadas em uma matriz de adjacência, onde a posição (i,j)(i,j) recebe o valor 1 se existe via para passagem de cabo e 0 caso contrário. A partir dessa matriz, uma matriz auxiliar de distâncias entre todos os pares de vértices fisicamente conectados é pré-computada, evitando o recálculo repetido da fórmula de distância (Euclidiana ou Haversine, conforme a instância) ao longo das gerações.

A população inicial é composta por 100 indivíduos. Para gerar cada indivíduo, a árvore é construída adotando o mecanismo de fronteira do algoritmo de Prim em versão randomizada: partindo da OLT (raiz), catalogam-se as arestas viáveis que ligam os vértices já visitados àqueles ainda não conectados, impedindo estritamente que clientes finais sejam utilizados como origem de novas conexões. Diferentemente de um sorteio puramente uniforme entre os candidatos, a escolha do próximo vértice segue uma loteria ponderada por posto: as kk arestas de menor distância recebem maior peso de seleção, de modo que o vértice mais próximo tem maior probabilidade de ser escolhido sem que os demais sejam descartados por completo. O valor de kk varia discretamente entre os indivíduos da população (entre 4 e 8), garantindo diversidade estrutural mesmo com o viés construtivo em direção a soluções mais curtas. A construção é interrompida assim que todos os vértices classificados como clientes finais estão conectados à rede.

Em seguida, aplica-se uma rotina de poda inspirada na Árvore de Steiner [Oliveira 2019], que percorre a estrutura de forma iterativa e elimina arestas de postes com grau unitário e sem filhos dependentes — ramificações que não atendem a nenhum cliente adicional. A aptidão (*fitness*) do indivíduo corresponde ao somatório das distâncias entre cada vértice ativo e seu respectivo nó antecessor após a poda. Antes de ingressar na população, cada indivíduo é submetido a uma verificação de viabilidade estrutural (conectividade de todo cliente até a OLT, ausência de ciclos e de arestas inexistentes na malha) e a um controle de duplicidade por hash, que descarta indivíduos estruturalmente idênticos a outros já presentes na geração, reforçando a diversidade genética desde a inicialização.

O ciclo evolutivo executa 1000 gerações com elitismo simples, no qual o melhor indivíduo encontrado até o momento é sempre preservado na geração seguinte. Para a seleção dos indivíduos reprodutores, comparam-se dois operadores: (I) Seleção por Torneio, em que três indivíduos são sorteados aleatoriamente e o de menor distância vence o embate; e (II) Seleção por Roleta, baseada em probabilidade proporcional ao inverso da distância total de cada indivíduo.

No cruzamento, o filho é reconstruído a partir da OLT sob uma política de prioridade genética em três níveis: arestas presentes em ambos os pais têm prioridade de herança (70% de chance), seguidas por arestas presentes em apenas um dos pais (25%), e, por fim, arestas inexistentes em qualquer dos pais, extraídas diretamente da matriz de adjacência como mecanismo de reparo estrutural (5%). Em cada um desses níveis, a aresta efetivamente incorporada é sorteada entre as candidatas mais curtas daquela categoria, e não uniformemente ao acaso — o mesmo princípio de loteria ponderada usado na inicialização. Quando uma aresta herdada de um pai é aceita, o algoritmo ainda tenta copiar um pequeno bloco de descendência (1 a 3 vértices) da mesma subárvore paterna, preservando fragmentos estruturais inteiros em vez de reconstruir a topologia aresta por aresta.

A mutação opera em dois níveis: primeiro, sorteia-se se o indivíduo será submetido à mutação (taxa configurável entre 0% e 20%); em caso positivo, cada gene ativo da árvore (isto é, cada vértice conectado) tem chance independente de sofrer alteração de seu nó antecessor (taxa configurável entre 1% e 5%). O novo antecessor deve satisfazer cinco condições: existência de via física na matriz de adjacência; pertencer à árvore atualmente ativa; ser distinto do antecessor atual; não ser um cliente final; e não formar ciclo, verificado por rastreamento recursivo dos antecessores até a OLT. Entre os candidatos válidos, prevalece novamente a preferência ponderada pela menor distância. Após cruzamento e mutação, o indivíduo é submetido a nova rodada de poda e validação estrutural antes de integrar a geração seguinte; caso a combinação de pais sorteada não produza, em até cinco tentativas, um filho válido e geneticamente distinto dos demais já gerados, aceita-se a última tentativa válida como forma de garantir o avanço do ciclo evolutivo.

Uma observação importante: nessa versão eu **não usei a palavra "convergência"** pra descrever o critério de parada — deixei explícito que o algoritmo roda um número fixo de 1000 gerações com elitismo, porque é isso que o código realmente faz (o limite de estagnação está configurado tão alto que na prática nunca é acionado). Se seus dados da Tabela 1/2 realmente pararam por

convergência antecipada em algum ponto, me avisa que eu ajusto essa frase — mas do jeito que está, essa versão é defensável sem ressalvas.

Faça em latex só

Drafting a methodology section in LaTeX format.

Drafting a methodology section in LaTeX format.

latex

```
\section{Metodologia}
```

A execução inicia com a leitura dos dados da instância, registrando as coordenadas (x, y) e a classificação dos vértices (OLT, postes e clientes). As vias viáveis são mapeadas em uma matriz de adjacência, onde a posição (i, j) recebe o valor 1 se existe via para passagem de cabo e 0 caso contrário. A partir dessa matriz, uma matriz auxiliar de distâncias entre todos os pares de vértices fisicamente conectados é pré-computada, evitando o recálculo repetido da fórmula de distância (Euclidiana ou Haversine, conforme a instância) ao longo das gerações.

A população inicial é composta por 100 indivíduos. Para gerar cada indivíduo, a árvore é construída adotando o mecanismo de fronteira do algoritmo de Prim em versão randomizada: partindo da OLT (raiz), catalogam-se as arestas viáveis que ligam os vértices já visitados àqueles ainda não conectados, impedindo estritamente que clientes finais sejam utilizados como origem de novas conexões. Diferentemente de um sorteio puramente uniforme entre os candidatos, a escolha do próximo vértice segue uma loteria ponderada por posto: as k arestas de menor distância recebem maior peso de seleção, de modo que o vértice mais próximo tem maior probabilidade de ser escolhido sem que os demais sejam descartados por completo. O valor de k varia discretamente entre os indivíduos da população (entre 4 e 8), garantindo diversidade estrutural mesmo com o viés construtivo em direção a soluções mais curtas. A construção é interrompida assim que todos os vértices classificados como clientes finais estão conectados à rede.

Em seguida, aplica-se uma rotina de poda inspirada na Árvore de Steiner \cite{Oliveira2019}, que percorre a estrutura de forma iterativa e elimina arestas de postes com grau unitário e sem filhos dependentes -- ramificações que não

atendem a nenhum cliente adicional. A aptidão (\textit{fitness}) do indivíduo corresponde ao somatório das distâncias entre cada vértice ativo e seu respect

A resposta do Claude foi interrompida.